Name: Basma Mohamed Mostafa

Group: SE3

Lab 4 Assignment

1. **Source code for both services and Dockerfiles:**

- **Product Service Folder:**

app.py:

```
from flask import Flask, jsonify
app = Flask(__name__)
# In-memory product catalog [cite: 105]
PRODUCTS = {
    1: {"id": 1, "name": "Laptop", "price": 1200},
    2: {"id": 2, "name": "Phone", "price": 650},
    3: {"id": 3, "name": "Headphones", "price": 120}
}
@app.route("/health", methods=["GET"])
def health():
    # Cloud-native health check endpoint [cite: 110]
    return jsonify({"service": "product-service", "status": "up"}), 200
@app.route("/products/<int:product_id>", methods=["GET"])
def get_product(product_id):
    # Product lookup API [cite: 113]
    product = PRODUCTS.get(product_id)
    if product:
```

```
        return jsonify(product), 200
    return jsonify({"error": "Product not found"}), 404
if __name__ == "__main__":
    # Container listens on all interfaces (0.0.0.0) [cite: 100]
    app.run(host="0.0.0.0", port=5001)
```

requirements.txt:

```
flask==3.0.3
```

Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5001
CMD ["python", "app.py"]
```

- **Order Service Folder:**

app.py:

```
import os
import requests
from flask import Flask, jsonify, request


app = Flask(__name__)
```

```
# Config via Environment variable (12-Factor App Principle #3) [cite: 149]
PRODUCT_SERVICE_URL = os.getenv("PRODUCT_SERVICE_URL", "http://product-service:5001")


@app.route("/health", methods=["GET"])
def health():
    return jsonify({"service": "order-service", "status": "up"}), 200


@app.route("/orders", methods=["POST"])
def create_order():
    data = request.get_json()
    product_id = data.get("product_id")
    quantity = data.get("quantity")


    # Call product-service [cite: 150]
    try:
        response = requests.get(f"{PRODUCT_SERVICE_URL}/products/{product_id}", timeout=2)
        if response.status_code == 200:
            product = response.json()
            total_price = product["price"] * quantity
            return jsonify({
```

```
            "message": "Order created",
            "product": product["name"],
            "total_price": total_price
        }), 201
    return jsonify({"error": "Product not found"}), 404
  except requests.exceptions.RequestException:
    return jsonify({"error": "product-service unavailable"}), 503
if __name__ == "__main__":
  app.run(host="0.0.0.0", port=5002)
```

requirements.txt:

```
flask==3.0.3
requests==2.32.3
```

Dockerfile:

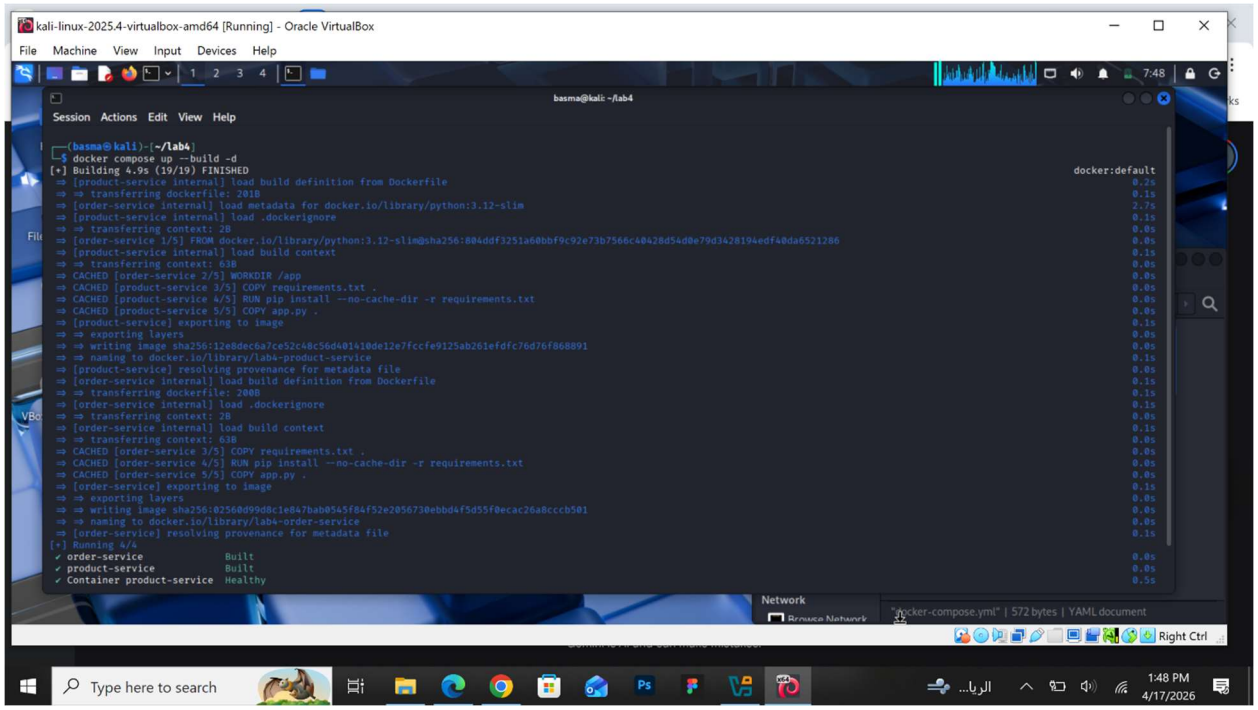
```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5002
CMD ["python", "app.py"]
```

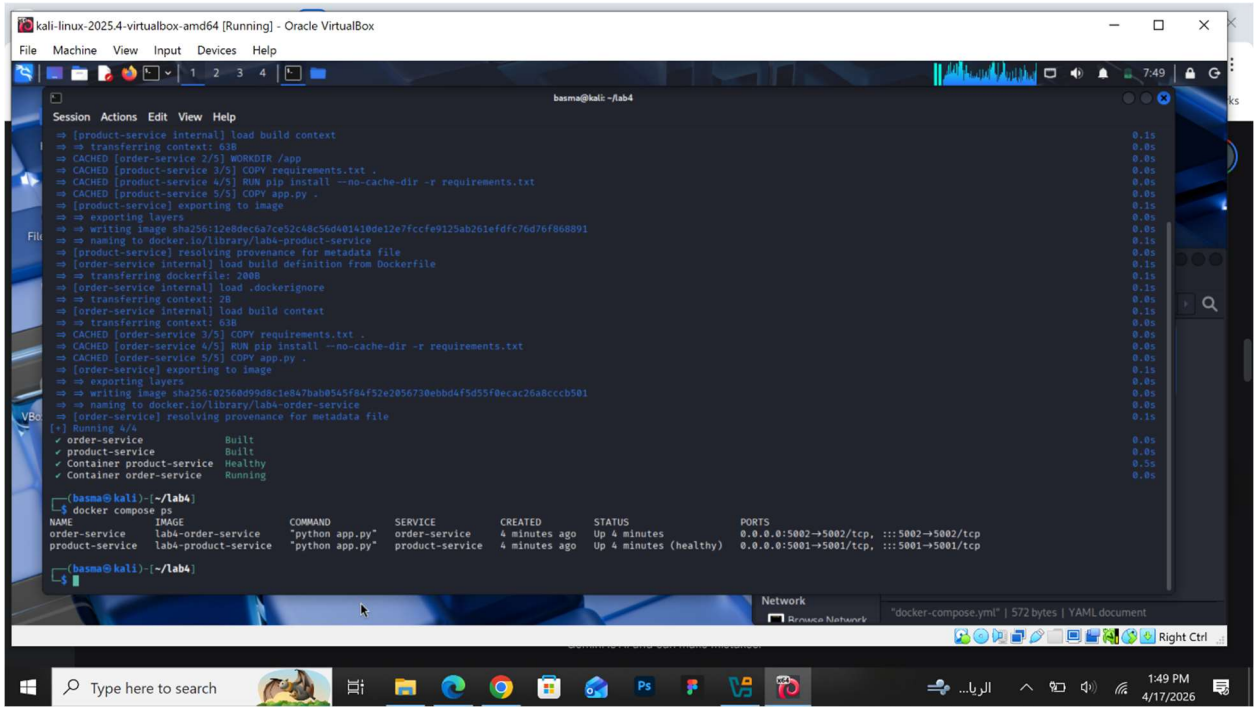
2. **docker-compose.yml file:**

```
services:
  product-service:
    build: ./product-service
    container_name: product-service
    ports:
      - "5001:5001"
    healthcheck:
      test: ["CMD", "python3", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:5001/health')"]
      interval: 10s
      timeout: 5s
      retries: 3

  order-service:
    build: ./order-service
    container_name: order-service
    ports:
      - "5002:5002"
    environment:
      - PRODUCT_SERVICE_URL=http://product-service:5001
    depends_on:
      product-service:
        condition: service_healthy
```

3. **Screenshots of successful API calls and failure simulation:**





kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

basma@kali: ~/lab4

Session Actions Edit View Help

```
┌──(basma㉿kali)-[~/lab4]
└─$ curl http://localhost:5001/products/1
{"id":1,"name":"Laptop","price":1200}

┌──(basma㉿kali)-[~/lab4]
└─$
```

Network

"docker-compose.yml" | 572 bytes | YAML document

Right Ctrl

Type here to search

29°C 1:53 PM 4/17/2026

kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

basma@kali: ~/lab4

Session Actions Edit View Help

```
┌──(basma㉿kali)-[~/lab4]
└─$ curl http://localhost:5001/products/1
{"id":1,"name":"Laptop","price":1200}

┌──(basma㉿kali)-[~/lab4]
└─$ curl -X POST http://localhost:5002/orders \
-H "Content-Type: application/json" \
-d '{"product_id": 1, "quantity": 2}'
{"message":"Order created","product":"Laptop","total_price":2400}

┌──(basma㉿kali)-[~/lab4]
└─$
```

Network

"docker-compose.yml" | 572 bytes | YAML document

Right Ctrl

Type here to search

الريا... 1:54 PM 4/17/2026

kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

basma@kali: ~/lab4

Session Actions Edit View Help

```
┌──(basma㉿kali)-[~/lab4]
└─$ curl http://localhost:5001/products/1
{"id":1,"name":"Laptop","price":1200}

┌──(basma㉿kali)-[~/lab4]
└─$ curl -X POST http://localhost:5002/orders \
-H "Content-Type: application/json" \
-d '{"product_id": 1, "quantity": 2}'
{"message":"Order created","product":"Laptop","total_price":2400}

┌──(basma㉿kali)-[~/lab4]
└─$ docker stop product-service
product-service

┌──(basma㉿kali)-[~/lab4]
└─$ curl -X POST http://localhost:5002/orders \
-H "Content-Type: application/json" \
-d '{"product_id": 1, "quantity": 1}'
{"error":"product-service unavailable"}

┌──(basma㉿kali)-[~/lab4]
└─$
```

Network

"docker-compose.yml" | 572 bytes | YAML document

Right Ctrl

Type here to search

1:55 PM
4/17/2026

kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

basma@kali: ~/lab4

Session Actions Edit View Help

```
┌──(basma㉿kali)-[~/lab4]
└─$ curl http://localhost:5001/products/1
{"id":1,"name":"Laptop","price":1200}

┌──(basma㉿kali)-[~/lab4]
└─$ curl -X POST http://localhost:5002/orders \
-H "Content-Type: application/json" \
-d '{"product_id": 1, "quantity": 2}'
{"message":"Order created","product":"Laptop","total_price":2400}

┌──(basma㉿kali)-[~/lab4]
└─$ docker stop product-service
product-service

┌──(basma㉿kali)-[~/lab4]
└─$ curl -X POST http://localhost:5002/orders \
-H "Content-Type: application/json" \
-d '{"product_id": 1, "quantity": 1}'
{"error":"product-service unavailable"}

┌──(basma㉿kali)-[~/lab4]
└─$ docker compose logs order-service
order-service  |  * Serving Flask app 'app'
order-service  |  * Debug mode: off
order-service  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
order-service  |  * Running on all addresses (0.0.0.0)
order-service  |  * Running on http://127.0.0.1:5002
order-service  |  * Running on http://172.20.0.3:5002
order-service  | Press CTRL+C to quit
order-service  | 172.20.0.1 - - [17/Apr/2026 11:53:58] "POST /orders HTTP/1.1" 201 -
order-service  | 172.20.0.1 - - [17/Apr/2026 11:55:15] "POST /orders HTTP/1.1" 503 -

┌──(basma㉿kali)-[~/lab4]
└─$
```

Network

"docker-compose.yml" | 572 bytes | YAML document

Right Ctrl

Type here to search

1:56 PM
4/17/2026

kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

basma@kali: ~/lab4

Session Actions Edit View Help

```
┌──(basma㉿kali)-[~/lab4]
└─$ curl -X POST http://localhost:5002/orders \
-H "Content-Type: application/json" \
-d '{"product_id": 1, "quantity": 2}'
{"message":"Order created","product":"Laptop","total_price":2400}

┌──(basma㉿kali)-[~/lab4]
└─$ docker stop product-service
product-service

┌──(basma㉿kali)-[~/lab4]
└─$ curl -X POST http://localhost:5002/orders \
-H "Content-Type: application/json" \
-d '{"product_id": 1, "quantity": 1}'
{"error":"product-service unavailable"}

┌──(basma㉿kali)-[~/lab4]
└─$ docker compose logs order-service
order-service  |  * Serving Flask app 'app'
order-service  |  * Debug mode: off
order-service  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
order-service  |  * Running on all addresses (0.0.0.0)
order-service  |  * Running on http://127.0.0.1:5002
order-service  |  * Running on http://172.20.0.3:5002
order-service  | Press CTRL+C to quit
order-service  | 172.20.0.1 - - [17/Apr/2026 11:53:58] "POST /orders HTTP/1.1" 201 -
order-service  | 172.20.0.1 - - [17/Apr/2026 11:55:15] "POST /orders HTTP/1.1" 503 -

┌──(basma㉿kali)-[~/lab4]
└─$ docker start product-service
product-service

┌──(basma㉿kali)-[~/lab4]
└─$ curl http://localhost:5001/products/1
{"id":1,"name":"Laptop","price":1200}

┌──(basma㉿kali)-[~/lab4]
└─$
```

Network

"docker-compose.yml" | 572 bytes | YAML document

Right Ctrl

Type here to search

1:57 PM
4/17/2026

4. **A short report (1-2 pages) answering the reflection questions:**

1-Which parts of this lab show the benefits of microservices over a monolith?

One advantage of microservices over monolithic applications that was observed in this lab includes separating the product-service and order-service from each other. The two services were running independently in their own containers which meant that changes in one would not affect the other.

Another advantage seen in this lab included the fact that when one service such as the product-service failed to start up, it did not affect the order-service and vice versa, something known as fault isolation. This is in contrast to a monolithic application that crashes completely if there are any issues in any of its components.

2- Which new complexities were introduced by splitting the system into two services?

Dividing the system brought about certain complications. To begin with, the services will interact through the network, using HTTP. Therefore, there will be the need for network errors, timeouts, and retry mechanisms.

Secondly, there was the complication of service discovery, whereby the order-service must figure out how to contact the product-service (Docker services). Moreover, debugging became a challenge since we will no longer have a single application but many services and logs.

In general, managing the system will be complicated as opposed to a monolithic approach.

3- What would break if network latency increased or one service became slow?

Network latency might affect the communication process when there are delays in response time or if one service performs poorly. For example, our application's order service relies on the product service; thus, if the product service operates slowly, the order service will be slowed down.

There is a risk of failure or timeouts that could overload the system further. In extreme situations, customers would not be able to place any orders. It proves that microservices are vulnerable to network latency due to their interdependence.

4- Which 12-factor app principles are visible in this implementation?

There are several 12-factor app concepts that have been used in this lab. First, there is one code base for each service as well as all the requirements that have been defined in requirement.txt.

Second, there is a configuration through the use of environment variables, and this ensures that the app is flexible. Third, the services are stateless, and therefore there is no local data storage, making them restartable without any loss of information.

Fourth, there is the concept of port binding, and the product-service is the backing service for the order-service.